Computers & Operations Research 172 (2024) 106824

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

New heuristics for the 2-stage assembly scheduling problem with total earliness and tardiness minimisation: A computational evaluation

Carla Talens [a,*], Jorge M.S. Valente [b], Victor Fernandez-Viagas [a,c]

[a] Industrial Management, School of Engineering, University of Seville, Camino de los Descubrimientos s/n, 41092 Seville, Spain
[b] LIAAD - INESC TEC, Faculdade de Economia da Universidade do Porto, Rua Dr. Roberto Frias, s/n, 4200-464 Porto, Portugal
[c] Instituto de Biomedicina de Sevilla, IBiS/Hospital Universitario Virgen Macarena/CSIC/Universidad de Sevilla, Spain

ARTICLE INFO

Keywords:
Scheduling
Assembly
Earliness
Tardiness
Just in time
Heuristics

ABSTRACT

Traditionally, scheduling literature has focused mainly on solving problems related to processing jobs with non-assembly operations. Despite the growing interest in the assembly literature in recent years, knowledge of the problem is still in its early stages in many aspects. In this regard, we are not aware of any previous contributions that address the assembly scheduling problem with just-in-time objectives. To fill this gap, this paper studies the 2-stage assembly scheduling problem minimising the sum of total earliness and total tardiness. We first analyse the relationship between the decision problem and the generation of the due dates of the jobs, and identify the equivalences with different related decision problems depending on the instances. The properties and conclusions obtained in the analysis are applied to design two constructive heuristics and a composite heuristic. To evaluate our proposals, different heuristics from the state-of-the-art of related scheduling problems are adapted, and a computational evaluation is carried out. The excellent behaviour of the proposed algorithms is demonstrated by an extensive computational evaluation.

1. Introduction

Nowadays, most companies and industries manufacture products formed by different components, which are processed in a first stage and then assembled into final products in a later stage. In the scheduling literature, this layout is known as the 2-stage assembly scheduling problem and, because of its relevance in real manufacturing, is receiving more attention from the research community. Thus, in recent years, an important number of works have been published on this problem (see e.g. Lee and Bang, 2016; Aydilek et al., 2017, and Lee, 2020), addressing in some of these publications real cases of industrial applications of the problem (see e.g. Lee et al., 1993; Potts et al., 1995; Cheng and Wang, 1999; Hwang and Lin, 2012; Fattahi et al., 2013; Liao et al., 2015; Allahverdi and Aydilek, 2015).

Regarding the objectives for this decision problem, the makespan minimisation (see e.g. Zhang et al., 2010) and the total completion time minimisation (see e.g. Sung and Kim, 2008; Al-Anzi and Allahverdi, 2012; Allahverdi and Al-Anzi, 2012; Al-Anzi and Allahverdi, 2013) are the most commonly studied. However, delivering products on time while minimising inventory is also important for companies, and none of the above objectives fits into a just-in-time policy. This approach has become an important goal for companies, as it reduces final inventory and work-in-process, improves the efficiency of the production system, and makes companies more flexible to customer demand. In fact, supply chain performance problems can be caused by early delivery or tardy delivery of products (Schaller and Valente, 2019). All these aforementioned aspects explain the increasing interest in the last decades in studying the penalisation of earliness and tardiness in other scheduling layouts (see e.g. Lauff and Werner, 2004; Józefowska, 2007; Shabtay and Steiner, 2012).

However, to the best of our knowledge, the 2-stage assembly scheduling problem with just-in-time objective has not been addressed so far. To cover this opportunity, we focus on the 2-stage assembly scheduling problem with the minimisation of total earliness and total tardiness as objective, where the first stage is composed of m_1 dedicated parallel machines. This problem is denoted $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ following the notation proposed in Framinan et al. (2019) and can be easily proven to be NP-hard since it is a generalisation of the two-stage assembly flowshop scheduling problem with the objective of minimising total tardiness, which is NP-hard in the strong sense (Lee and Bang, 2016).

In addition, the problem under consideration is a generalisation of two well-known scheduling problems: the customer order scheduling problem ($DP_m \rightarrow 0$), when there is not any assembly machine; and the flowshop scheduling problem, when there is only one dedicated

* Corresponding author.
E-mail addresses: cartafa@us.es (C. Talens), jvalente@fep.up.pt (J.M.S. Valente), vfernandezviagas@us.es (V. Fernandez-Viagas).

https://doi.org/10.1016/j.cor.2024.106824
Received 5 March 2024; Received in revised form 24 June 2024; Accepted 26 August 2024
Available online 29 August 2024
0305-0548/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

machine and one assembly machine (i.e., can be denoted as $DP_1 \rightarrow 1$). With regard to the literature from these related problems, there are some contributions considering due-date-related objectives. Regarding the customer order scheduling problem, Wagneur and Sriskandarajah (1993) and Leung et al. (2006) showed that the EDD rule is optimal for the objective of maximum lateness and Leung et al. (2006) also proposed a heuristic, called GHM, for minimising the number of tardy jobs. Lin and Kononov (2007) also studied this objective and designed a heuristic, called LAK. Wagneur and Sriskandarajah (1993) demonstrated that the problem with total tardiness is unary NP-hard and Lee (2013) presented a B&B procedure and three constructive heuristics, named TPT-EDD, MPT-EDD, and OMDD, for the problem. Framinan and Perez-Gonzalez (2018) designed a fast constructive heuristic, denoted FAP, and a matheuristic. Finally, Xu et al. (2016) proposed some dominance rules, a B&B procedure, and two metaheuristics for the problem with position-based learning effect. Regarding the 2-stage assembly scheduling problem with one assembly machine, it has been addressed with different due-date-related objectives. Allahverdi and Al-Anzi (2006) addressed the $DP_m \rightarrow 1 \| L_{\max}$ using metaheuristics. Al-Anzi and Allahverdi (2006) and Al-Anzi and Allahverdi (2007) studied the same objective with sequence-independent setups also applying metaheuristics. The $DP_m \rightarrow 1 \| \sum T_j$ was addressed by Allahverdi and Aydilek (2015) and Lee and Bang (2016). The first work proposed an insertion algorithm, PIA, and several metaheuristics, while the latter proposed a MILP, a B&B procedure and a two-phase heuristic, called mNEH-PI. Allahverdi et al. (2016a) studied the problem with sequence-independent setups and proposed several metaheuristics. The total weighted tardiness objective was studied by Lee (2020), who proposed four heuristics, HEDD, HWT, WSPT_MAX and WSPT_AVER. Aydilek et al. (2017) addressed the objective of maximum tardiness with sequence-independent setups, studied some properties of the problem, and proposed a metaheuristic. Additionally, the $DP_m \rightarrow 1 \| \sum U_j$ problem was studied by Allahverdi et al. (2016b) using metaheuristics.

Finally, regarding the flowshop scheduling problem, Fernandez-Viagas et al. (2016a) proposed a constructive heuristic, named FDF, and some composite heuristics for the objective of minimising total earliness and tardiness, and Schaller and Valente (2019) designed two heuristics, LIN1 and LIN2, for the same objective and also implemented two heuristics, SLK and SLK/P, adapted from Panwalkar and Iskander (1977).

Thus, given the importance of just-in-time objectives and the scarcity of studies and analysis of the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem, the contribution of this paper is fourfold:

- A detailed analysis of the relationship between the generation of the due dates of the jobs and the equivalences of the problem under study with other related decision problems is conducted.
- Two new constructive heuristics are developed to efficiently solve the problem addressed in this work. The first constructive heuristic, CH1, iteratively constructs a solution by using specific knowledge of the problem and the second, CH2, applies the same procedure as CH1 and adds a partial local search procedure at each iteration.
- A composite heuristic is proposed to improve the quality of the solutions obtained by the second constructive heuristic. This method, LR_CH2, constructs and evaluates several sequences instead of only one with the aim of improving the quality of the solution obtained.
- An exhaustive computational experimentation is carried out, which shows that the generated due dates represent the problem under study and that the proposed heuristics outperform the heuristics adapted from the state-of-the-art of the related problems.

To deal with this contribution, the rest of the paper is organised as follows. In Section 2, the problem is analysed in detail: in Section 2.1 we formally describe the problem; we state some properties of the problem in Section 2.2, and in Section 2.3, we present a procedure to study the relationship between the decision problem and the generation of the due dates of the jobs. In Section 3, we explain the proposed heuristics: the two constructive heuristics in Sections 3.1 and 3.2; and in Section 3.3 the composite heuristic. Next, computational experimentation is performed in Section 4: we explain the set of instances and performance indicators in Section 4.1; we conduct the analysis of due dates and their suitability in Section 4.2 and perform the calibration of the parameters in Section 4.3; in Section 4.4 we explain the algorithms implemented from the literature, and the comparison of heuristics is carried out in Section 4.5. Finally, in Section 5 we present the main conclusions and some future research lines.

2. Analysis of the problem

In this section, we analyse in detail the problem studied in this contribution. First, in Section 2.1, we present the formal description of the problem; then, in Section 2.2, we state two theoretical properties of the problem, and finally, in Section 2.3, we present the procedure for identifying the decision problem depending on the due dates of the jobs.

2.1. Problem description

In this paper, the problem studied can be defined as follows: There are n jobs to be scheduled on a two-stage layout. Each job has $m_1 + 1$ non-resumable operations. The first m_1 operations are processed in the first stage composed of m_1 dedicated machines (i.e. machines can only perform one and only one type of operation), while the last operation has to be processed in any of the m_2 identical parallel machines of the assembly stage (where components are assembled). This assembly operation can be started only after all m_1 operations have been completed. In Fig. 1, the configuration of the two stages is shown, where $O^1_{i,j}$ denotes the operation of the component $i \in \{1, \dots, m_1\}$ of job j in the first stage and O^2_j denotes the assembly operation of job j. The processing time of each job $j \in \{1, 2, \dots, n\}$ on each machine i in the first stage is p_{ij}, the assembly processing time in the second stage is at_j and the due date is d_j. All jobs are available for being processed at time zero (release times are negligible). Unforced idle time is not allowed to be inserted between jobs, i.e. the forced idle time that is allowed is that between the first and second stage when the assembly machine is available, but the components are not ready to be assembled. The decision problem consists of scheduling the jobs in the two stages with the objective of minimising the sum of total earliness and total tardiness. Given a sequence of jobs $\Pi = (\Pi[1], \dots, \Pi[n])$, the earliness and tardiness of a job j can be computed from its completion time as follows. Let $C1_j$ denote the maximum completion time of the job j in the first stage. Consequently, $C1_{\Pi[j]}$ is the maximum completion time of job in position $[j]$ in sequence Π, which is denoted as $C1_{[j]}$ when it is not misleading. It is computed as follows:

$$C1_{\Pi[j]} = C1_{[j]} = \max_{i=1,\dots,m_1} \left\{ \sum_{k=1}^{j} p_{i[k]} \right\} \tag{1}$$

Additionally, $\lambda_{[j]}$ denotes the assembly machine where the job in position $[j]$ is assigned. Then, the completion times on the second stage can be computed considering this assignment $\lambda_{[j]}$ of the jobs to the assembly machines. The completion time of each job in position $[j]$ is computed according to Eq. (2), where the variable $C_{\lambda_{[j]},[j]}$ denotes the completion time in machine $\lambda_{[j]}$ of the jobs in the sequence until position $[j]$:

$$C_{\lambda_{[j]},[j]} = \max\{C_{\lambda_{[j]},[j-1]}, C1_{[j]}\} + at_{[j]} \ \forall j \tag{2}$$

where $C_{\lambda_{[j]},0} = 0$. Then, $C_{[j]}$ is the completion time of the job processed in position $[j]$ of the sequence and can be computed by Eq. (3).

$$C_{[j]} = C_{\lambda_{[j]},[j]} \ \forall j \tag{3}$$

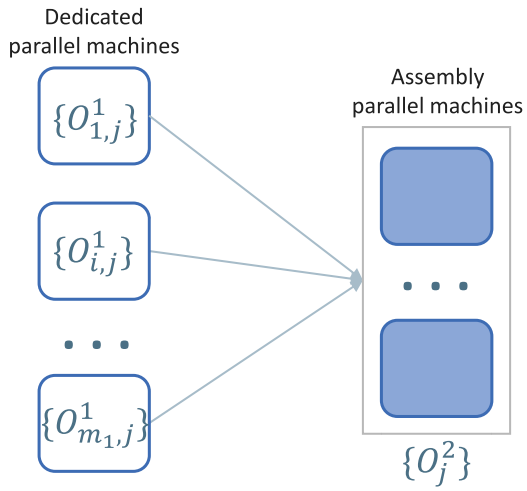


Fig. 1. $DP_m \rightarrow P_m$ layout.

The earliness and tardiness of job j in sequence Π are defined as $E_j = \max\{d_j - C_j, 0\}$ and $T_j = \max\{C_j - d_j, 0\}$, respectively. Finally, total earliness and total tardiness is defined as $\sum E_j + \sum T_j = \sum_{\forall j} \max\{d_j - C_j, 0\} + \sum_{\forall j} \max\{C_j - d_j, 0\}$.

2.2. Theoretical results

According to the description of the problem presented in the previous section and based on the analysis conducted by Fernandez-Viagas and Framinan (2015a) on related scheduling problems, the due dates of jobs in a particular instance could potentially have a high influence on the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem (also denoted $\mathcal{P}_a$). On the basis of this evidence, the following two properties can be stated to analyse the relationship with related scheduling problems if due dates are extremely tight or loose.

Property 2.1. *If $d_j \geq WM$ $\forall j$, being WM the worst (maximum) makespan for a specific instance s of the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem, then an optimal solution for s is obtained by solving the corresponding $DP_m \rightarrow P_m \| - \sum C_j$ problem.*

Proof. According to Property 2.1 of Fernandez-Viagas and Framinan (2015a), since each due date is greater than or equal to the worst makespan WM, each due date d_j is greater than or equal to its completion time, $C_j(\Pi)$, i.e. $d_j \geq WM \geq C_j(\Pi)$, $\forall j$. Then, minimising $\sum_{\forall j} \max\{d_j - C_j(\Pi), 0\} + \sum_{\forall j} \max\{C_j(\Pi) - d_j, 0\} = \sum_{\forall j}(d_j - C_j(\Pi)) + 0 = \sum_{\forall j} d_j - \sum_{\forall j} C_j(\Pi) = const - \sum_{\forall j} C_j(\Pi)$.

Property 2.2. *If $d_j \leq \max_{i=1,\ldots,m_1}\{\sum_{k=1}^{j} p_{i[k]}\} + at_j$ $\forall j$ and for a specific instance s of the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$. Then, an optimal solution for s is obtained by solving the corresponding $DP_m \rightarrow P_m \| \sum C_j$ problem.*

Proof. Following a similar reasoning as Property 2.2 of Fernandez-Viagas and Framinan (2015a), each completion time $C_j(\Pi)$ ($\forall$ Π) is greater than or equal to its due date, since $d_j \leq \max_{i=1,\ldots,m_1}\{\sum_{k=1}^{j} p_{i[k]}\} + at_j$ $\forall j$,. Then, $\sum_{\forall j} \max\{d_j - C_j(\Pi), 0\} + \sum_{\forall j} \max\{C_j(\Pi) - d_j, 0\} = 0 + \sum_{\forall j}(C_j(\Pi) - d_j) = \sum_{\forall j} C_j(\Pi) - \sum_{\forall j} d_j = \sum_{\forall j} C_j(\Pi) - const$.

By the statement of these two properties, we formalise the relationship between the type of the optimisation problem and the due dates and the processing times of an instance. It can be stated that if the due dates are extremely loose, the problem consists of maximising the flowtime, denoted $\mathcal{P}_b$, which is trivial, according to Property 2.1, while the problem is similar to extremely the flowtime minimisation problem when the due dates are tight, denoted $\mathcal{P}_c$, according to Property 2.2.

2.3. Instances analysis procedure

According to the previous results, in this section, we propose a procedure to analyse the similarities among the three problems, $\mathcal{P}_a$, $\mathcal{P}_b$ and $\mathcal{P}_c$. This procedure compares the best objective function found for the previous problems by applying the Iterated Greedy (IG) algorithm, developed by Ruiz and Stützle (2007). More specifically, the procedure is as follows:

Step 1. Analyse how tight or loose the due dates of instance s are. The indicator υ defined in Eq. (4) is computed (see Fernandez-Viagas and Framinan, 2015a for more details) for instance s. Regarding this indicator, when $\upsilon \sim 0$, i.e. the due dates are tight and minimising total earliness and tardiness is similar to minimising total flowtime. Therefore, algorithms designed to solve the $DP_m \rightarrow P_m \| \sum C_j$ problem would have good performance. On the contrary, when $\upsilon \sim 1$, i.e. the due dates are loose and minimising total earliness and tardiness is similar to maximising total flowtime ($DP_m \rightarrow P_m \| - \sum C_j$ problem). It can be shown that $\upsilon \in [0, 1]$.

$$\upsilon = \sum_{j=1}^{n} \frac{\min\{1; \max\{0; \upsilon_j\}\}}{n} = \sum_{j=1}^{n} \frac{\min\{WM - p_j; \max\{0; d_j - p_j\}\}}{n \cdot (\mathrm{WM} - p_j)} \tag{4}$$

where υ_j is computed for each job j according to the next equation:

$$\upsilon_j = \frac{d_j - p_j}{WM - p_j} \tag{5}$$

Step 2. Solve instance s by IG algorithm modifying the original objective function, makespan, to minimise total earliness and total tardiness. Let IG_{OF} denote this version of the IG algorithm and Π_{OF} denote the best sequence obtained by IG_{OF}.

Step 3. Solve instance s by IG algorithm modifying the original objective function, makespan, to minimise total flowtime, denoted $IG_{\mathrm{FT_{min}}}$. Let $\Pi_{\mathrm{FT_{min}}}$ denote the best sequence obtained by $IG_{\mathrm{FT_{min}}}$.

Step 4. Solve instance s by IG algorithm modifying the original objective function, makespan, to maximise total flowtime, denoted $IG_{\mathrm{FT_{max}}}$. Let $\Pi_{\mathrm{FT_{max}}}$ denote the best sequence obtained by $IG_{\mathrm{FT_{max}}}$.

Step 5. Compute the total earliness and total tardiness of Π_{OF}, $\Pi_{\mathrm{FT_{min}}}$ and $\Pi_{\mathrm{FT_{max}}}$ for each instance s, denoted $OF_s(IG_{OF})$, $OF_s(IG_{\Pi_{\mathrm{FT_{min}}}})$ and $OF_s(IG_{\Pi_{\mathrm{FT_{max}}}})$, respectively.

Step 6. Compute Worst_s and Best_s as the worst and best known total earliness and tardiness, respectively, in instance s, among $OF_s(IG_{OF})$, $OF_s(IG_{\Pi_{\mathrm{FT_{min}}}})$, $OF_s(IG_{\Pi_{\mathrm{FT_{max}}}})$ and the different values of the objective function from the computational evaluation performed in Section 4.5.

Step 7. Analyse the performance of $IG_{\mathrm{FT_{min}}}$ and $IG_{\mathrm{FT_{max}}}$ in solving instance s. We compute the Compared Relative Deviation Index ($CRDI$) defined in Eq. (6) (see Fernandez-Viagas and Framinan, 2015a for more details). This indicator, $CRDI_{hl} \in [-100, 100]$, is the difference between the Relative Deviation Index, RDI_{hs}, detailed in Section 4.1, of the algorithm h ($IG_{\mathrm{FT_{min}}}$ or $IG_{\mathrm{FT_{max}}}$) and that of the IG_{OF} when both algorithms are applied to instance s. When $CRDI_{hl} \sim 100$, the performance of the algorithm h ($IG_{\mathrm{FT_{min}}}$ or $IG_{\mathrm{FT_{max}}}$) is much worse than that of IG_{OF}; when $CRDI_{hl} \sim 0$, the performance of both methods is similar; and when $CRDI_{hl} \sim -100$, the performance of the algorithm i is much better than that of IG_{OF}.

$$CRDI_{hl} = RDI_{hs} - RDI_{IG_{OF}} = \frac{OF_{hs} - OF_{IG_{OF}}}{Worst_s - Best_s} \cdot 100 \tag{6}$$

Step 8. Repeat Steps 1–7 until all instances are evaluated.

3. Constructive and composite heuristics

In this section, we design three heuristics for the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem. In Section 3.1, we present the first constructive heuristic, denoted CH1, which obtains a solution by iteratively adding jobs at the end of the sequence. Then, in Section 3.2, we propose the second constructive heuristic, denoted CH2, by adding a partial local search procedure after scheduling a job in each iteration of CH1. Finally, in Section 3.3, we present a composite heuristic, denoted LR_CH2, which constructs and evaluates x (partial) sequences by applying the CH2 procedure.

3.1. Proposed constructive heuristic CH1

As briefly explained above, the CH1 heuristic iteratively constructs a sequence by evaluating all unscheduled jobs and selecting the most appropriate job to be added to the end of the partial sequence. This procedure has also been proposed to solve other assembly problems (see e.g. Framinan and Perez-Gonzalez, 2017; Talens et al., 2022) and scheduling problems with due-date-related objectives (see e.g. Fernandez-Viagas et al., 2016b; Framinan and Perez-Gonzalez, 2017) with excellent results. The design of an index Ψ that adequately represents the suitability of an unscheduled job to be scheduled is essential for the efficiency of this type of algorithms, since once a job is scheduled, we cannot change its position in the next iterations.

$CH1$ starts with a partial sequence Π with a single job, which is the job with the lowest value of $E_{j,0}$, i.e. the earliness when scheduled in the first position of Π, breaking ties in favour of the job with the lowest value of $IT_{j,0}$. $IT_{j,0}$ is the amount of time that the assembly machine is waiting while all components of job j are processed in the first position of each dedicated parallel machine, computed as $IT_{j,0} = \max_{i \in \{1,\dots,m_1\}} p_{i[j]}$. Let $\mathcal{W}_0$ the set of all unscheduled jobs excluding the first job scheduled in Π. In each iteration ($k \in \{2, \dots, n\}$), CH1 evaluates each unscheduled job $\omega_{lk} \in (l = 1, \dots, n-k+1)$ in the set of unscheduled jobs until iteration k, i.e. $\mathcal{W}_k$, as a candidate to be scheduled in the last position of Π. Then, the heuristic selects the job from $\mathcal{W}_k$ with the lowest value of $\Psi_{\omega_{lk}}$, adds it to the end of the partial sequence Π_k in iteration k and obtains the partial sequence Π_{k+1} of the next iteration. The algorithm repeats these steps for $k-2$ iterations. In our case, the design of the index Ψ incorporates the results of the previous section, taking into account the relationship with the related scheduling problems depending on the due dates. As seen in Section 2, we can deal with three different situations depending on due dates: tight due dates and the $DP_m \rightarrow P_m \| \sum C_j$ problem; loose due dates and the $DP_m \rightarrow P_m \| -\sum C_j$ problem; and normal, i.e. neither loose nor tight, due dates and the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem. Thus, the heuristic would check which of these situations is verified at each iteration:

- Case 1. If the due dates are tight, we deal with the $DP_m \rightarrow P_m \| \sum C_j$ problem: In Talens et al. (2020), the authors designed a constructive heuristic that also iteratively constructs a sequence. In this case, the jobs were scheduled according to an index that minimises the weighted idle time and the completion time of the candidate jobs. This index has been adapted and is computed by Eq. (7):

$$\Psi\omega_{lk} = (N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) + \alpha \cdot C_{\omega_{lk}}(\Pi_k) \tag{7}$$

  where $IT_{\omega_{lk}}(\Pi_k)$ is:

$$IT_{\omega_{lk}}(\Pi_k) = \max\{C1_{\omega_{lk}}(\Pi_k) - C_{[k-1]}(\Pi_k), 0\} \tag{8}$$

  being $C1_{\omega_{lk}}(\Pi_k)$ the completion time of ω_{lk} in the first stage and $C_{[k-1]}(\Pi_k)$ is the completion time in the second stage of the last scheduled job before ω_{lk}.
  As a consequence, in CH1, the suitability of a job to be scheduled is computed with this index when due dates are tight. More specifically, we propose that this index is applied if there are at least four tardy jobs, denoted NT_k, and the fraction of tardy jobs in the partial sequence is equal to or greater than a, i.e. $(NT_k > 3)$ and $(NT_k/(n-k) \geq a)$, where a is a parameter of the algorithm calibrated in Section 4.3.
- Case 2. If the due dates are loose, we deal with the $DP_m \rightarrow P_m \| -\sum C_j$ problem: Depending on the looseness of the due dates, this case is divided into three subcases.
  - Case 2.1. All due dates are extremely high: The problem is similar to maximise total flowtime. As maximising total flowtime is not a common objective, there are no algorithms that address this objective. Thus, we compute the inverse index for the first case, $\Psi\omega_{lk} = -\Psi^1_{\omega_{lk}}$. In this case, we define NE_k as the number of jobs in $\mathcal{W}_k$ whose earliness is higher than $(n-k) \cdot c$, being c a parameter to control how early the jobs are. This index is computed, according to Eq. (9), only when the following conditions are met: There are at least four candidates jobs, i.e. $(n-k > 3)$, all the candidates in iteration k are early, which is fulfilled if $C_{\omega_{lk}}(\Pi_k) < d_{\omega_{lk}}$, $\forall j$, and $NE_k = n-k$.

$$\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k) \tag{9}$$

  - Case 2.2. Only some of the due dates are extremely high: When due dates are not so loose, we add the earliness $E_{\omega_{lk}}$ of the candidate job to the index of the previous subcase. This index is applied when condition $b \cdot (n-k) \leq NE_k < n-k$ is verified, being b and β parameters of the algorithm calibrated in Section 4.3.

$$\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k) + \beta \cdot E_{\omega_{lk}}(\Pi_k) \tag{10}$$

  - Case 2.3. Only a few due dates are extremely high: As in the previous case, the due dates are not so loose, but we only consider the earliness $E_{\omega_{lk}}$ of the candidate job in the index, since it implicitly also takes into account tardy jobs and selects them first since their earliness is equal to zero. This index is applied when condition $d \cdot (n-k) \leq NE_k < b \cdot (n-k)$ is verified, being d a parameter of the algorithm calibrated in Section 4.3.

$$\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k) \tag{11}$$

- Case 3. If the due dates are neither loose nor tight, we deal with the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem and, consequently, the heuristic should minimise total earliness and tardiness. In this case, the algorithm computes the index $\Psi\omega_{lk}$, according to Eq. (12), for each candidate job, and adds to the end of the partial sequence the job with the lowest value of $\Psi\omega_{lk}$.

$$\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k) + \gamma \cdot IT_{\omega_{lk}}(\Pi_k) \tag{12}$$

In this algorithm, the parameters a, b, c, d, α, β and γ need to be tuned (see Section 4.3). The complexity of CH1 is $O((n-1) \cdot [(n-k) \cdot (m_1+m_2)]) \sim O(n^2 \cdot (m_1+m_2))$, since $n-1$ iterations are performed in the main loop and $n-k$ jobs are evaluated in each iteration, computing the completion time and idle time, so all machines m_1+m_2 are evaluated. The pseudocode of CH1 is provided in Algorithm 1.

3.2. Proposed constructive heuristic CH2

The second constructive heuristic, labelled CH2, is designed on the basis of CH1 due to its good performance. This heuristic is composed of two stages: First, we schedule a job according to the index Ψ, explained in the previous section, and then perform an improvement mechanism with the objective of evaluating the neighbour solutions of the partial sequence. This improvement procedure interchanges the last scheduled job, denoted σ_k, with its $\min\{k-1, z \cdot n\}$ previous jobs, being z a percentage. In this heuristic, in a specific iteration k, we consider a relative

Algorithm 1 Pseudo-code of the proposed heuristic $CH1$

procedure CH1
  All jobs are initially unscheduled;
  Determination of $IT_{j,0}$, $C_{j,0}$ and $E_{j,0}$;
  π_1:= Job with lowest value of $E_{j,0}$, breaking ties in favour of the job with the lowest value of $IT_{j,0}$;
  $\Pi_1 = (\pi_1)$
  for $k = 2$ to $n-1$ **do**
    Determination of each $IT_{\omega_{lk}}(\Pi_k)$, $C_{\omega_{lk}}(\Pi_k)$, NT_k, NE_k and $E_{\omega_{lk}}(\Pi_k)$, $\forall j \in [1, n-k]$;
    if $NT_k > 3$ & $NT_k/(n-k) \geq a$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = (N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) + \alpha \cdot C_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $NE_k = n-k$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $b \cdot (n-k) \leq NE_k < n-k$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k) + \beta E_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $d \cdot (n-k) \leq NE_k < b \cdot (n-k)$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k)$
      end for
    else
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k) + \gamma \cdot IT_{\omega_{lk}}(\Pi_k)$
      end for
    end if
    $r := \text{argmin}_{1 \leq l \leq n-\lambda+1} \Psi\omega_{lk}$;
    Add ω_r at the end of Π_k obtaining Π_{k+1}, i.e. $\Pi_{k+1} := (\pi_1, \ldots, \pi_{k-1}, \omega_r)$;
    Extract ω_r from $\mathcal{W}$
  end for
  return $\sum_{\forall j \in \Pi}(E_j + T_j)$
end procedure

improvement strategy consisting in directly replacing the best solution when it is improved. After both the best solution has been replaced and the iteration k is not finished, the procedure continues to interchange σ_k with the remaining jobs in the new best solution. In the experiments carried out in Section 4.5, the levels of $z = \{10, 20, 30, 40, 50, 100\}\%$ are considered.

In Fig. 2, we show an example of the improvement procedure included in CH2 with ten jobs and $z = 40\%$. So, from iteration $k = 5$ to $k = n-1$, the last job added to the sequence is interchanged with its four previous jobs, since $\min\{k-1, zn\} = \{k-1, 0.4 \cdot 10\} = 4 \; \forall \; k \in \{5, n-1\}$, which in the case of iteration $k = 5$ matches up with all the previous jobs. Thus, in iteration $k = 5$, job 4 is added to the partial sequence $\Pi_5 = (5, 7, 6, 8)$, so the first evaluated sequence is $\Pi'_5 = (5, 7, 6, 8, 4)$ with $currsol = bestsol = 23$. Now, job 4 is interchanged with its four previous jobs. First, job 4 is interchanged with job 8 and the sequence $\Pi'_5 = (5, 7, 6, 4, 8)$ is evaluated with $currsol = 73$. Then, job 4 is interchanged with job 6 and the sequence $\Pi'_5 = (5, 7, 4, 8, 6)$ is obtained with $currsol = 72$. Next, job 4 is interchanged with job 7 and, finally, job 4 is interchanged with job 5. In iteration $k = 6$, job 3 is added to the partial sequence $\Pi_5 = (5, 7, 6, 8, 4)$, so the first evaluated sequence is $\Pi_6 = (5, 7, 6, 8, 4, 3)$ with $currsol = bestsol = 24$. Now, job 3 is interchanged with its four previous jobs. First, job 3 is interchanged with job 4, the sequence $\Pi'_6 = (5, 7, 6, 8, 3, 4)$ is evaluated, and its $currsol = 22$. As $currsol < bestsol$, then $bestsol = currsol$ and now job 4 is interchanged with job 8, and the sequence $\Pi'_6 = (5, 7, 6, 4, 3, 8)$ is evaluated with $currsol = 78$. Next, job 4 is interchanged with job 6 and with job 7, without improving $bestsol$. Therefore, for the next iteration, the sequence $\Pi_6 = (5, 7, 6, 8, 3, 4)$ is considered.

The pseudocode of CH1 is presented in Algorithm 2. The complexity of this heuristic is $O(n^3 \cdot (m_1 + m_2))$.

Algorithm 2 Pseudo-code of the proposed heuristic $CH2$

procedure CH2
  All jobs are initially unscheduled;
  Determination of $IT_{j,0}$, $C_{j,0}$ and $E_{j,0}$;
  π_1:= Job with lowest value of $E_{j,0}$, breaking ties in favour of the job with the lowest value of $IT_{j,0}$;
  $\Pi_1 = (\pi_1)$
  for $k = 2$ to $n-1$ **do**
    Determination of each $IT_{\omega_{lk}}(\Pi_k)$, $C_{\omega_{lk}}(\Pi_k)$, NT_k, NE_k and $E_{\omega_{lk}}(\Pi_k)$, $\forall j \in [1, n-k]$;
    if $NT_k > 3$ & $NT_k/(n-k) \geq a$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = (N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) + \alpha \cdot C_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $NE_k = n-k$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $b \cdot (n-k) \leq NE_k < n-k$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = -(N-k-2) \cdot IT_{\omega_{lk}}(\Pi_k) - \alpha \cdot C_{\omega_{lk}}(\Pi_k) + \beta \cdot E_{\omega_{lk}}(\Pi_k)$
      end for
    else if $NT_k > 3$ & $Allearlyjobs$ & $d \cdot (n-k) \leq NE_k < b \cdot (n-k)$ **then**
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k)$
      end for
    else
      for $j = 1$ to $n-k$ **do**
        $\Psi\omega_{lk} = E_{\omega_{lk}}(\Pi_k) + \gamma \cdot IT_{\omega_{lk}}(\Pi_k)$
      end for
    end if
    $r := \text{argmin}_{1 \leq l \leq n-\lambda+1} \Psi\omega_{lk}$;
    Add ω_r at the end of Π_k obtaining Π_{k+1}, i.e. $\Pi_{k+1} := (\pi_1, \ldots, \pi_{k-1}, \omega_r)$;
    Extract ω_r from $\mathcal{W}$
    Compute the completion times of Π_k.
    Do $bestsol = \sum E_j + \sum T_j \; (\Pi)$
    for $s = 1$ to $s = \{\min\{k-1, z \cdot n\}\}$ **do**
      Let Π' be the neighbour obtained interchanging ω_r with job in position s.
      Evaluate Π'. Do $currsol := \sum E_j + \sum T_j \; (\Pi')$
      if $currsol < bestsol$ **then** $bestsol = currsol$
        $\Pi = \Pi'$
      end if
    end for
  end for
return $\sum_{\forall j \in \Pi}(E_j + T_j)$
end procedure

3.3. Proposed composite heuristics

The composite heuristic, denoted LR_CH2, proposed in this section follows the idea of the heuristic proposed by Liu and Reeves (2001), called LR, and also applied by Liu and Reeves (2001), Pan and Ruiz (2013) and Fernandez-Viagas and Framinan (2015b). Basically, this method repeats x times the CH2 heuristic by changing the initial job to be inserted. More specifically, the procedure consists of the next steps:

1. Sort all jobs in non-decreasing order of $E_{j,0}$, breaking ties in favour of the job with the lowest $IT_{j,0}$, and obtain order U with all unscheduled jobs.

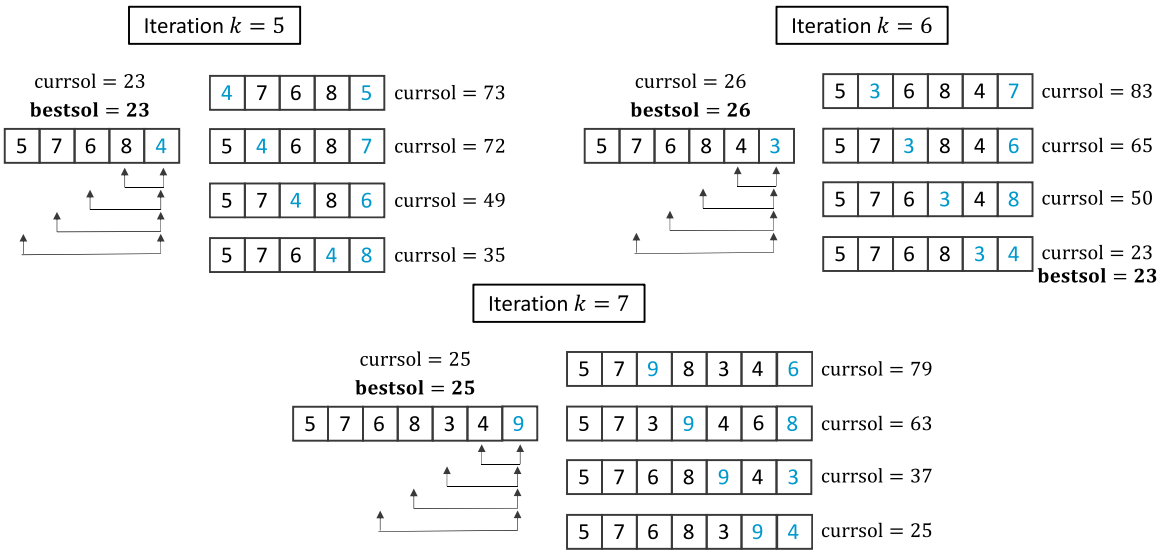


Fig. 2. Example of the procedure followed by heuristic $CH2$.

2. Define Π_x partial sequences, which are initially formed by a single job, placing each of the first x ranked job in the first position of each Π_x. Then construct iteratively x sequences by adding the remaining U_x jobs one by one using the heuristic CH2 procedure.
3. Among the x solutions obtained, choose the one with the minimum value of the objective function, i.e. total earliness and tardiness.

The number of sequences evaluated by this procedure is expected to largely influence the performance of LR_CH2. Thus, we evaluate the behaviour of this composite heuristic when x takes different values, e.g. $x = \{2, 5, 10, n/10, n/2, n\}$. Note that this procedure has also been applied to the CH1 heuristic in preliminary tests but has been discarded due to its low performance compared to the different versions of LR_CH2.

4. Computational evaluation

In this section, we describe the methodology for evaluating the proposed heuristics and the methods adapted from the related literature. Section 4.1 presents the testbed and the performance indicators required to perform the algorithm comparison. Section 4.2 analyses the suitability of the generated testbed, while Section 4.3 explains the design of experiments carried out to properly calibrate the parameters related to CH1. Section 4.4 briefly describes the algorithms adapted and implemented from the state-of-the-art of related problems. Finally, in Section 4.5, we perform the algorithms evaluation that leads us to identify the set of efficient heuristics. This comparison has been carried out using the same programming language (C# using Visual Studio), the same common functions and libraries, and on an Intel Core i7-3770 PC with 3.4 GHz and 16 GB RAM.

4.1. Set of instances and performance indicators

In this section, we present the set of instances used to evaluate the algorithms. This testbed is generated according to the procedure followed in Fernandez-Viagas et al. (2016b), Schaller and Valente (2019) and Talens et al. (2020) and is composed of five instances for each combination of $n = \{50, 150, 250, 350\}$, $m_1 = \{2, 4, 6, 8\}$, $m_2 = \{1, 2, 3\}$, $T = \{0.25, 0.5, 0.75\}$ and $R = \{0.2, 0.6, 1.0\}$, being T and R the parameters related to the mean and standard deviation of the due dates, respectively. We use the procedure described by Potts and Van Wassenhove (1982) to generate these due dates. Thereby, the due date of each job j is generated following a uniform distribution between $P \cdot (1 - T - R/2)$ and $P \cdot (1 - T + R/2)$, where P is a lower bound for the makespan, adapted from that proposed by Blocher and Chhajed (2008). Regarding the processing times of the jobs in the first stage, they are generated from a U[1, 100] distribution, while the processing times in the second stage are generated from a $U[1, m_2 \cdot 100]$ distribution to balance both stages. In total, this set of instances is made up of 1620 instances.

For comparison of algorithms, we compute different indexes to compare the quality of the solutions and the computational effort. Concerning the quality of the solutions of the different heuristics, the Average Relative Deviation Index, $ARDI$, is computed according to the following equation:

$$ARDI_h = \frac{\sum_{\forall s} RDI_{hs}}{S}, \ \forall s = 1, \ldots, S \tag{13}$$

being S the total number of instances and RDI_{hs} is calculated according to:

$$RDI_{hs} = \frac{OF_{hs} - Best_s}{Worst_l - Best_s} \tag{14}$$

where OF_{hj} is the total earliness and tardiness obtained by the heuristic h when it solves instance s. Concerning the computational effort, each heuristic h requires a time, in seconds, denoted T_{hs} to achieve a solution for instance s. Consequently, the Average CPU ($ACPU$) time can be computed according to:

$$ACPU_h = \frac{\sum_{\forall s} T_{hs}}{S} \tag{15}$$

Additionally, we also use the Relative Percentage Indicator (labelled RPT) due to the problems that the indicator $ACPU$ presents when used to compare heuristics with different stopping criteria (see Fernandez-Viagas and Framinan, 2015b; Talens et al., 2020). As we compare heuristics with different number of steps in their methodology, we compute the RPT indicator following Eq. (16).

$$RPT_{hs} = \frac{T_{hs}}{ACPU_s} \tag{16}$$

where $ACPU_s$ is the average CPU time required for solving instance s and is computed by Eq. (17):

$$ACPU_s = \frac{\sum_{\forall h} T_{hs}}{H} \tag{17}$$

Finally, the Average RPT ($ARPT$) is computed as follows:

$$ARPT_h = \sum_{\forall s}^{S} \frac{RPT_{hs}}{S} \tag{18}$$

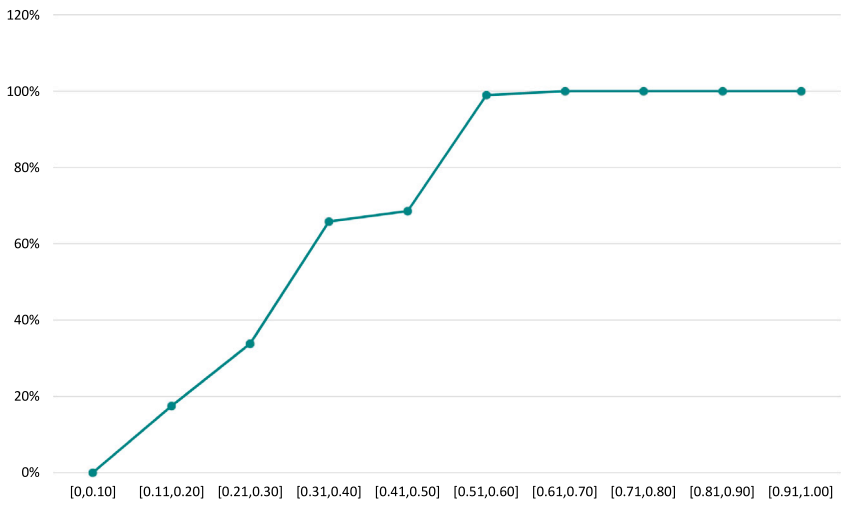


Fig. 3. Distribution of the cumulative percentage of instances depending on υ for the generated instances.

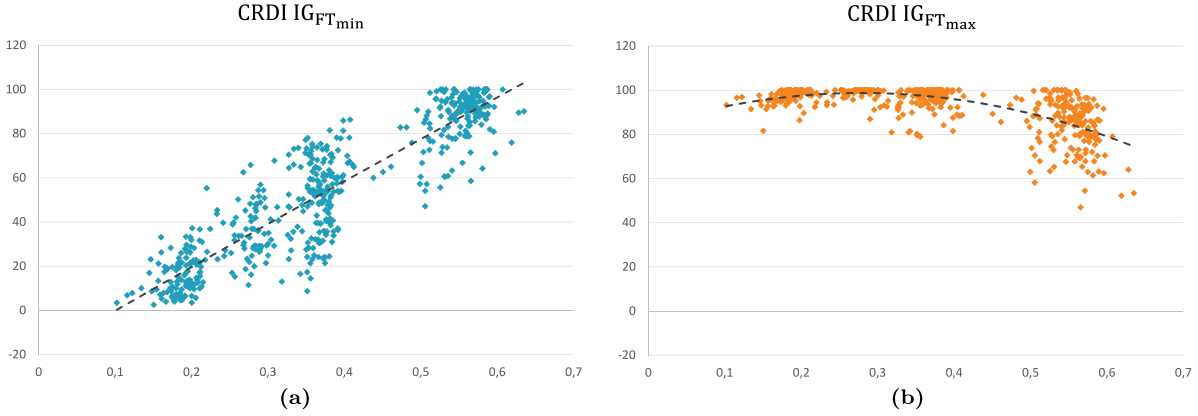


Fig. 4. $CRDI_{IG_{min}}$ and $CRDI_{IG_{max}}$ for different values of υ in each instance.

4.2. Suitability of the instances

The objective of this section is to discuss the similarities with other related decision problems by evaluating the instances generated in the previous section. Following the procedure presented in Section 2.3, we first analyse the dependence between due dates and processing times. To do so, we compute the indicator υ, according to Eq. (4), for each instance of the benchmark generated in this work (see Section 4.1), where the worst makespan, WM, is computed with a modified version of the Iterated Greedy algorithm to maximise makespan. Fig. 3 shows the amount of instances for different intervals of υ. Regarding the values of index υ obtained in these experiments, the minimum and maximum values are 0.1013 and 0.6495, respectively.

Next, we carry out the experiments necessary to analyse the instances of the problem following the procedure of Section 2. In Fig. 4 the distribution of the $CRDI_{ij}$ values of $IG_{\mathrm{FT_{min}}}$ and $IG_{\mathrm{FT_{max}}}$ versus the values of υ is shown. Regarding the similarity of the problem with the FT_{min} problem, if we solve the instances with $IG_{\mathrm{FT_{min}}}$, its $CRDI_{\mathrm{IG_{FT_{min}}}}$, the average $CRDI$ of $IG_{\mathrm{FT_{min}}}$ for all instances, is equal to 52.21, which is much higher than 0. Thus, the existing methods for the objective FT_{min} are not appropriate for the problem under study. Additionally, it can be observed that for low values of υ, on the one hand, the testbed does not consider instances with tight due dates, since the minimum value of υ is equal to 0.10 and, on the other hand, for instances with $\upsilon < 0.2$, $CRDI_{\mathrm{IG_{FT_{min}}}}$ is equal to 13.00.

Regarding the similarity of the problem with FT_{max} problem, if we solve the instances with $IG_{\mathrm{FT_{max}}}$, its $CRDI_{\mathrm{IG_{FT_{max}}}}$, the average $CRDI$ of $IG_{\mathrm{FT_{max}}}$ for all the instances, is equal to 92.35, which is much higher than 0. Thus, the existing methods for the objective FT_{max} are not appropriate for the problem considered in this work. In fact, its performance would be close to the worst solutions. Furthermore, it can be observed that the testbed does not consider instances with loose due dates. In fact, for instances with $\upsilon > 0.6$, $CRDI_{\mathrm{IG_{FT_{max}}}}$ is equal to 91.89.

It can be concluded that the proposed instances have neither tight nor loose due dates, and, in fact, these instances are the most challenging ones. Therefore, if the methods proposed in Section 3 achieve a good performance, it is because they solve the difficult instances well and would not have any trouble with the easy ones, where the priority index, explained in Section 3, is simple and more than appropriate.

4.3. Experimental parameter tuning

The constructive heuristic CH1 proposed in this work uses several parameters: a, b, c, d, α, β, and γ. Thus, we perform a full factorial design of experiments to appropriately define these parameters. The following values of the parameters are considered for the experiments: $a = \{0.5, 0.6, 0.7\}$, $b = \{0.7, 0.75, 0.8\}$, $c = \{15, 20, 25\}$, $d = \{0.65, 0.7, 0.75\}$, $\alpha = \{0.1, 0.2, 0.3\}$, $\beta = \{0.5, 1.5, 2.5\}$, $\gamma = \{2, 4, 6\}$.

To perform these experiments, we generate two instances for different values of n, m_1, m_2, T, and R following the procedure from Section 4.1. Note that to avoid overfitting these parameters, these instances, 648 in total, must be different from those used in Section 4.5.

The combination of parameters a, b, c, d, α, β and γ that achieves the best performance is defined through a non-parametric Kruskal–Wallis,

which is performed after proving that the normality and homoscedasticity assumptions are not satisfied. The best value of $ARPD$ is obtained for $a = 0.6$, $b = 0.8$, $c = 15$, $d = 0.75$, $\alpha = 0.2$, $\beta = 1.5$ and $\gamma = 4$. The Tukey's post-hoc comparison method is also performed to pairwise compare all groups and significant differences between all groups are found.

4.4. Implemented algorithms

To determine the performance of the algorithms proposed in Section 3, they have been compared with 16 heuristics adapted from the related problems reviewed in Section 1, i.e. the customer order scheduling problem, the 2-stage assembly scheduling problem with only one machine in the second stage, and the flowshop scheduling problem. The following heuristics are chosen for the comparison:

- Heuristics from $DPm \rightarrow 1||\sum T_j$. These heuristics are adapted by assigning each job to the first available machine in the second stage when the completion times in this stage are computed.
  - mNEH-PI (Lee and Bang, 2016): this heuristic applies the NEH algorithm to the EDD sequence and then, to improve the solution, interchanges all jobs two by two.
  - PIA (Allahverdi and Aydilek, 2015): this heuristic inserts all the jobs into all the positions of an initial sequence, obtains the best sequence, and then interchanges all the jobs in adjacent positions.
  - H_EDD, H_SPT_MAX and H_SPT_AVER (Lee, 2020), proposed for the particular objective of $\sum w_j T_j$: these heuristics sort the jobs in nondecreasing order of d_j, $\sum_{i=1,\ldots,m_1} p_{ij} + at_j$ and $(1/m_1)\sum_{i=1,\ldots,m_1} p_{ij} + at_j$, respectively.
- Heuristics from $DPm \rightarrow 0||\sum T_j$
  - TPT_EDD, MPT_EDD and OMDD (Lee, 2013): TPT_EDD is adapted by computing the indicator $\bar{p}_j = \sum_{i=1,\ldots,m_1} p_{ij} + at_j$ and $\bar{d}_j = \max\{\bar{p}_j, (m_1 \cdot m_2)d_j\}$ for each order $j \in \{1,\ldots,n\}$. Then, all jobs are sequenced in nondecreasing order of $\bar{d}_j$ values. To adapt MPT_EDD the indicator $\bar{p}_j = \max_{i=1,\ldots,m_1} p_{ij} + at_j$ is computed. Finally, OMDD is adapted by computing $\alpha_j = \max\{C_j - C_\pi, d_j - C_\pi\}$ for each unscheduled job considering the second stage in its calculation, where C_π is the completion time of partial sequence π and C_j is the completion time of job j if it is appended at the end of π. At each iteration, the job with the lowest value of α_j is sequenced.
  - FAP (Framinan and Perez-Gonzalez, 2018): this heuristic computes $\eta_j = \max\{C_j(\pi_j) - d_j; 0\} + T^*(\pi_j)$ for each job j in each iteration and sequences the job with the lowest value of η_j. To adapt it, we consider the second stage and assign each job to the first available machine when $C_j(\pi_j)$ is computed, i.e. the completion time of the job j if it is added to the partial sequence π. In this indicator, $T^*(\pi_j)$, calculated considering the second stage, is an estimate of the contribution to the total earliness and total tardiness of the rest of the unscheduled jobs.
- Heuristics from $DPm \rightarrow 0||\sum U_j$. These heuristics are adapted by assigning each job to the first available machine in the second stage when the completion times are computed.
  - GHM (Leung et al., 2006): This heuristic obtains a sequence with two parts. The first part, π_e, is formed by jobs completed on time, and the second part of the sequence, π_l, by jobs not completed on time.
  - LAK (Lin and Kononov, 2007): The jobs are initially sorted according to the EDD rule, and each time a job is tardy, the job with the largest value of the adapted indicator $\bar{p}_j = \max_{i=1,\ldots,m_1} p_{ij} + at_j$ is removed.
- Heuristics from $Fm||\sum E_j + \sum T_j$.
  - SLK and SLK_P (Panwalkar and Iskander, 1977): In the minimum slack (SLK) rule the job with the minimum slack, $s_j = d_j - C_{jM}$, is selected in each iteration. In the minimum slack per required time (SLK/P) the job with the minimum value of the ratio $SLK/P_j = s_j/P_j$ is selected in each iteration, where P_j is the total time between the start and finish of the job j if it is added to the end of the sequence. To adapt these heuristics, we consider the problem as a flowshop with two stages when C_{jM}, s_j, and P_j are computed.
  - FDF (Fernandez-Viagas et al., 2016a): this heuristic iteratively constructs a sequence by scheduling the job with the lowest value of an indicator, whose calculation depends on the number of tardy jobs and the number of early jobs, among others, in each iteration. To adapt it, we consider the second stage by assigning each job to the first available machine when the completion times are computed.
  - LIN1 and LIN2 (Schaller and Valente, 2019): these two heuristics schedule, at each iteration, the job with the largest value of an indicator that depends on the slack s_j of each job, considering the second stage in its calculation.

4.5. Comparison of heuristics

In this section, we determine the effectiveness of the proposed heuristics CH1, CH2, and LR_CH2 by comparing them with the heuristics presented in the previous section. Table 1 shows the average results in terms of RDI, CPU, and RPT with the group of efficient heuristics marked in bold. The average results are also graphically shown in Fig. 5, where the y-axis represents the $ARDI$ for each heuristic and the x-axis represents the $ARPT$. Taking into account these results, we can draw the following conclusions:

- The group of heuristics H_EDD, H_SPT_MAX, H_SPT_AVER, TPT_EDD, MPT_EDD, and OMDD, which could be considered as dispatching rules, obtain a solution very quickly, but the quality is quite low. Note that these methods are not displayed in Fig. 5 for a clearer interpretation of the results. Among these methods, the one with the minimum $ARDI$ is MPT_EDD.
- CH1 ($ARPD = 11.808$) clearly outperforms the heuristics SLK ($ARPD = 33.094$) and SLK_P ($ARPD = 27.822$) using less computational effort, and obtains a better value of $ARDI$ than FDF ($ARPD = 16.885$), LIN1 ($ARPD = 37.767$) and LIN2 ($ARPD = 23.783$) with a similar computational time. From these results, it seems that CH1 obtains good performance by scheduling only one job at each iteration and, thus, requiring less CPU time.
- CH2 (z=10%) ($ARPD = 6.483$), CH2 (z=20%) ($ARPD = 4.054$), CH2 (z=30%) ($ARPD = 3.245$) and CH2 (z=40%) ($ARPD = 2.939$) outperform GHM, FAP, nMEH-PI and PIA with $ARPD$ equal to 32.647, 60.235, 28.171 and 79.785, respectively, using less computational time.
- CH2 (z=50%) ($ARPD = 2.795$) and CH2 (z=100%) ($ARPD = 2.595$) outperform FAP, nMEH-PI, and PIA using less computational time.
- Regarding the composite heuristic LR_CH2, whose range of $ARDI$ varies from 1.990 to 0.09, it gets an interesting improvement as the value of x increases, but also considerably increases computational times. However, the computational effort is worthwhile due to the good $ARDI$ results.

Therefore, we can conclude that the proposed heuristic $CH1$ and the variants of $CH2$ are efficient in solving the problem. More specifically, these heuristics show very good performance, achieving an important reduction in the value of $ARDI$ and competitive computational time. Finally, with respect to the composite heuristic LR_CH2 and its

Table 1
Summary of result of the different heuristics. The efficient frontier is marked in bold.

Heuristic	*ARDI*	*ACPU*	*ARPT*	Heuristic	*ARDI*	*ACPU*	*ARPT*
MPT_EDD	**32.767**	**0.000**	**0.000**	PIA	79.785	20.215	3.388
TPT_EDD	32.779	0.000	0.000	CH1	**11.808**	**0.009**	**0.006**
H_EDD	33.098	0.000	0.000	**CH2 (z=10%)**	**6.483**	**0.101**	**0.022**
H_SPT_AVER	93.151	0.000	0.000	**CH2 (z=20%)**	**4.054**	**0.189**	**0.036**
H_SPT_MAX	93.120	0.000	0.000	**CH2 (z=30%)**	**3.245**	**0.273**	**0.050**
LAK	56.313	0.003	0.002	**CH2 (z=40%)**	**2.939**	**0.352**	**0.063**
OMDD	84.077	0.004	0.002	**CH2 (z=50%)**	**2.795**	**0.422**	**0.075**
FDF	16.885	0.009	0.005	**CH2 (z=100%)**	**2.595**	**0.605**	**0.104**
LIN1	37.767	0.016	0.006	**LR_CH2 (x=2)**	**1.990**	**1.206**	**0.205**
LIN2	23.783	0.016	0.006	**LR_CH2 (x=5)**	**1.316**	**3.008**	**0.511**
SLK	33.094	0.030	0.010	**LR_CH2 (x=10)**	**0.950**	**6.010**	**1.022**
SLK_P	27.822	0.031	0.010	**LR_CH2 (x=15)**	**0.752**	**9.009**	**1.532**
GHM	32.647	0.389	0.069	**LR_CH2 (x = n/10)**	**0.860**	**18.778**	**1.444**
MEHD	83.404	0.581	0.098	**LR_CH2 (x = n/2)**	**0.260**	**93.826**	**7.212**
FAP	60.235	1.553	0.266	**LR_CH2 (x = n)**	**0.090**	**187.642**	**14.421**
mNEH_PI	28.171	2.609	0.432				

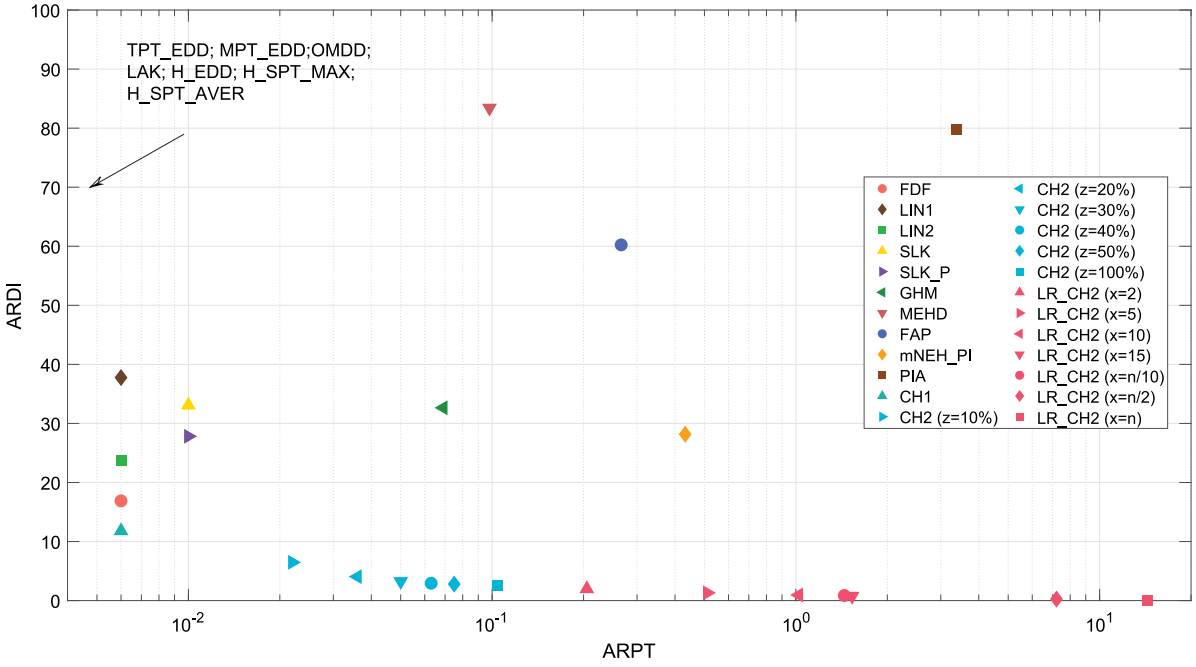


Fig. 5. *ARPD* versus *ARPT*. *ARPT* (x-axis) is shown in logarithmic scale.

variants, they achieve the best *ARDI*, but they are time-consuming heuristics. Thus, they can be the best methods when there are no computational time constraints. Finally, the efficient frontier is made up of the fast heuristic MPT_EDD and the proposed heuristics CH1, the different versions of CH2 with $z = \{10, 20, 30, 40, 50, 100\}\%$ and the different versions of LR_CH2 with $x = \{2, 5, 10, n/10, n/2, n\}$.

A Holm's procedure (Holm, 1979) is carried out to verify the statistical significance of these results and determine the efficient frontier. This procedure evaluates each hypothesis using a nonparametric Mann–Whitney test, which assumes a confidence level of 0.95 to establish the p-value of each hypothesis considering the *ARDI* indicator as the response variable. In Holm's test, we sort the hypotheses in increasing order of the p-values obtained in the Mann–Whitney test. If $p \leq \alpha/(N - i + 1)$, being N the total number of hypotheses and i the hypothesis evaluated, there is no significant evidence to accept each hypothesis. Table 2 shows the results, where R means that the hypothesis is rejected by Holm's procedure and, as can be observed, all hypotheses are rejected. In this test, we compare the candidate heuristics to be part of the efficient frontier that are closer, in terms of *ARDI*, to a heuristic from the literature. Consequently, the efficient frontier is formed by heuristics MPT_EDD, CH1, CH2 with $z = \{10, 20, 30, 40, 50, 100\}\%$ and LR_CH2 with $x = \{2, 5, 10, n/10, n/2, n\}$.

5. Conclusions

This paper studies the 2-stage assembly scheduling problem with the objective of minimising total earliness and tardiness since, to the best of our knowledge, it has never been addressed so far and its importance in industry is increasing due to its real applications. Therefore, we first analyse in detail the decision problem through the study of the influence of the due dates generation on the decision problem and the discussion of the similarities with different related decision problems. Next, we propose new efficient heuristics to solve the problem under study. More specifically, we design two specific constructive heuristics considering the characteristics of the problem. The first heuristic, CH1, obtains a sequence by iteratively analysing how loose or tight the due dates are and adding a job to the end of a partial sequence, and the second heuristic, CH2, adds a local search after scheduling a job at each iteration. Furthermore, we have also proposed a composite heuristic LR_CH2, which consists of generating several sequences following the CH2 procedure, so that more solutions are evaluated. Finally, we perform an extensive computational evaluation to verify the good performance of the proposed heuristics with the most promising algorithms adapted from the literature of the related problems. In view of the computational results, it can be concluded

Table 2
Mann–Whitney's procedure with response variable $ARDI$.

i	H_i	p-value	Mann–Whitney	$\alpha/(k-i+1)$	Holm's procedure
1	CH1=FDF	<0.001	R	0.0125	R
2	CH2 (z=10)=GHM	<0.001	R	0.0167	R
3	LR_CH2 (x=2)=mNEH-PI	<0.001	R	0.0250	R
4	LR_CH2 (x=10)=PIA	<0.001	R	0.0500	R

that our proposal shows an excellent performance. In fact, the group of the most efficient heuristics is formed by the fast heuristic MPT_EDD and the proposed heuristics CH1, the different versions of CH2 with $z = \{10, 20, 30, 40, 50, 100\}\%$ and the different versions of LR_CH2 with $x = \{2, 5, 10, n/10, n/2, n\}$. Therefore, these methods can be considered the new state-of-the-art heuristics for the $DP_m \rightarrow P_m \| \sum E_j + \sum T_j$ problem.

Regarding future research lines, it can be addressed the proposal of metaheuristics to solve the problem, the study of other objectives, such as quadratic total earliness and total tardiness, the analysis of different policies to improve the factory's production indicators, e.g. inventory level or rate of manufactured products, or the study of the influence of the unforced idle time on the earliness.

CRediT authorship contribution statement

Carla Talens: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Jorge M.S. Valente:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Victor Fernandez-Viagas:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Funding acquisition.

Data availability

Data will be made available on request.

Acknowledgements

The authors wish to thank the referees for their comments on the earlier versions of the manuscript. This research has been funded by the Spanish Ministry of Science and Innovation under project "ENSURE" with reference PID2022-142062OB-I00. In addition, this study has been funded by Instituto de Salud Carlos III (ISCIII) through the project "PI22/01096" and co-funded by the European Union.

References

Al-Anzi, F.S., Allahverdi, A., 2006. Computer assembly scheduling problem. In: 2006 International Conference on Service Systems and Service Management, Vol. 2. IEEE, pp. 1188–1191.
Al-Anzi, F.S., Allahverdi, A., 2007. A self-adaptive differential evolution heuristic for two-stage assembly scheduling problem to minimize maximum lateness with setup times. European J. Oper. Res. 182 (1), 80–94.
Al-Anzi, F.S., Allahverdi, A., 2012. Better heuristics for a two-stage multi- machine assembly scheduling problem to minimize total completion time better heuristics for a two-stage multi-machine assembly scheduling problem to minimize total completion time. Int. J. Oper. Res. (Taichung) 9, 66–75.
Al-Anzi, F.S., Allahverdi, A., 2013. An artificial immune system heuristic for two-stage multi-machine assembly scheduling problem to minimize total completion time. J. Manuf. Syst. 32 (4), 825–830.
Allahverdi, A., Al-Anzi, F.S., 2006. A PSO and a Tabu search heuristics for the assembly scheduling problem of the two-stage distributed database application. Comput. Oper. Res. 33 (4), 1056–1080.
Allahverdi, A., Al-Anzi, F., 2012. A new heuristic for the queries scheduling problem on distributed database systems to minimize mean completion time. In: Proceedings of the 21st International Conference on Software Engineering and Data Engineering. SEDE 2012.
Allahverdi, A., Aydilek, H., 2015. The two stage assembly flowshop scheduling problem to minimize total tardiness. J. Intell. Manuf. 26 (2), 225–237.
Allahverdi, A., Aydilek, A., Aydilek, H., 2016a. Minimizing the number of tardy jobs on a two-stage assembly flowshop. J. Ind. Prod. Eng. 33 (6), 391–403.
Allahverdi, A., Aydilek, H., Aydilek, A., 2016b. Two-stage assembly scheduling problem for minimizing total tardiness with setup times. Appl. Math. Model. 40 (17–18), 7796–7815.
Aydilek, A., Aydilek, H., Allahverdi, A., 2017. Minimising maximum tardiness in assembly flowshops with setup times. Int. J. Prod. Res. 55 (24), 7541–7565.
Blocher, J.D., Chhajed, D., 2008. Minimizing customer order lead-time in a two-stage assembly supply chain. Ann. Oper. Res. 161 (1), 25–52.
Cheng, T.E., Wang, G., 1999. Scheduling the fabrication and assembly of components in a two-machine flowshop. IIE Trans. 31 (2), 135–143.
Fattahi, P., Hosseini, S.M.H., Jolai, F., 2013. A mathematical model and extension algorithm for assembly flexible flow shop scheduling problem. Int. J. Adv. Manuf. Technol. 65 (5), 787–802.
Fernandez-Viagas, V., Dios, M., Framinan, J.M., 2016a. Efficient constructive and composite heuristics for the permutation flowshop to minimise total earliness and tardiness. Comput. Oper. Res. 75, 38–48.
Fernandez-Viagas, V., Framinan, J.M., 2015a. NEH-based heuristics for the permutation flowshop scheduling problem to minimise total tardiness. Comput. Oper. Res. 60, 27–36.
Fernandez-Viagas, V., Framinan, J.M., 2015b. A new set of high-performing heuristics to minimise flowtime in permutation flowshops. Comput. Oper. Res. 53, 68–80.
Fernandez-Viagas, V., Leisten, R., Framinan, J.M., 2016b. A computational evaluation of constructive and improvement heuristics for the blocking flow shop to minimise total flowtime. Expert Syst. Appl. 61, 290–301.
Framinan, J.M., Perez-Gonzalez, P., 2017. The 2-stage assembly flowshop scheduling problem with total completion time: Efficient constructive heuristic and metaheuristic. Comput. Oper. Res. 88, 237–246.
Framinan, J.M., Perez-Gonzalez, P., 2018. Order scheduling with tardiness objective: Improved approximate solutions. European J. Oper. Res. 266 (3), 840–850.
Framinan, J.M., Perez-Gonzalez, P., Fernandez-Viagas, V., 2019. Deterministic assembly scheduling problems: A review and classification of concurrent-type scheduling models and solution procedures. European J. Oper. Res. 273, 401–417.
Holm, S., 1979. Board of the foundation of the scandinavian journal of statistics 6(2). pp. 65–70.
Hwang, F.J., Lin, B.M., 2012. Two-stage assembly-type flowshop batch scheduling problem subject to a fixed job sequence. J. Oper. Res. Soc. 63 (6), 839–845.
Józefowska, J., 2007. Just-In-Time Scheduling: Models and Algorithms for Computer and Manufacturing Systems. Springer.
Lauff, V., Werner, F., 2004. Scheduling with common due date, earliness and tardiness penalties for multimachine problems: A survey. Math. Comput. Modelling 40 (5), 637–655.
Lee, I.S., 2013. Minimizing total tardiness for the order scheduling problem. Int. J. Prod. Econ. 144 (1), 128–134.
Lee, I., 2020. A scheduling problem to minimize total weighted tardiness in the two-stage assembly flowshop. Math. Probl. Eng. 2020, 1–10.
Lee, J.-Y., Bang, J.-Y., 2016. A two-stage assembly-type flowshop scheduling problem for minimizing total tardiness. Math. Probl. Eng. 2016, 1–10.
Lee, C.-Y., Cheng, T.C.E., Lin, B.M.T., 1993. Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. Manage. Sci. 39 (5), 616–625.
Leung, J.Y.-T., Li, H., Pinedo, M., 2006. Scheduling orders for multiple product types with due date related objectives. European J. Oper. Res. 168 (2), 370–389, Feature Cluster on Mathematical Finance and Risk Management.
Liao, C.J., Lee, C.H., Lee, H.C., 2015. An efficient heuristic for a two-stage assembly scheduling problem with batch setup times to minimize makespan. Comput. Ind. Eng. 88 (313), 317–325.
Lin, B., Kononov, A., 2007. Customer order scheduling to minimize the number of late jobs. European J. Oper. Res. 183 (2), 944–948.
Liu, J., Reeves, C.R., 2001. Constructive and composite heuristic solutions to the $P//\sum C_i$ scheduling problem. European J. Oper. Res. 132 (2), 439–452, Data Envelopment Analysis.
Pan, Q.K., Ruiz, R., 2013. A comprehensive review and evaluation of permutation flowshop heuristics to minimize flowtime. Comput. Oper. Res. 40 (1), 117–128.
Panwalkar, S.S., Iskander, W., 1977. A survey of scheduling rules. Oper. Res. 25 (1), 45–61.
Potts, C.N., Sevast'janov, S.V., Strusevich, V.A., Van Wassenhove, L.N., Zwaneveld, C.M., 1995. The two-stage assembly scheduling problem: Complexity and approximation. Oper. Res. 43 (2), 346–355.
Potts, C., Van Wassenhove, L., 1982. A decomposition algorithm for the single machine total tardiness problem. Oper. Res. Lett. 1 (5), 177–181.
Ruiz, R., Stützle, T., 2007. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. European J. Oper. Res. 177 (3), 2033–2049.

Schaller, J., Valente, J.M., 2019. Heuristics for scheduling jobs in a permutation flow shop to minimize total earliness and tardiness with unforced idle time allowed. Expert Syst. Appl. 119, 376–386.

Shabtay, D., Steiner, G., 2012. Scheduling to maximize the number of just-in-time jobs: A survey, vol. 60, pp. 3–20.

Sung, C.S., Kim, H.A., 2008. A two-stage multiple-machine assembly scheduling problem for minimizing sum of completion times. Int. J. Prod. Econ. 113 (2), 1038–1048.

Talens, C., Fernandez-Viagas, V., Perez-Gonzalez, P., Costa, A., 2022. Constructive and composite heuristics for the 2-stage assembly scheduling problem with periodic maintenance and makespan objective. Expert Syst. Appl. 206.

Talens, C., Fernandez-Viagas, V., Perez-Gonzalez, P., Framinan, J.M., 2020. New efficient constructive heuristics for the two-stage multi-machine assembly scheduling problem. Comput. Ind. Eng. 140 (October 2019), 106223.

Wagneur, E., Sriskandarajah, C., 1993. Openshops with jobs overlap. European J. Oper. Res. 71 (3), 366–378.

Xu, J., Wu, C.-C., Yin, Y., Zhao, C., Chiou, Y.-T., Lin, W.-C., 2016. An order scheduling problem with position-based learning effect. Comput. Oper. Res. 74, 175–186.

Zhang, Y., Zhou, Z., Liu, J., 2010. The production scheduling problem in a multi-page invoice printing system. Comput. Oper. Res. 37 (10), 1814–1821.